

研读几十个热门 Skills 之后，我总结出了这十个写好 Skill 的套路

 mp.weixin.qq.com/s/BBJ3aLi0WxzUh1DM0_ytwa

三元同学

Original

最近在开发 Skill 的时候（包括之前文章提到的 code-review-expert skill），为了写好 Skill，其实我在之前把 skills.sh 上排名靠前的几十个 Skill 全翻了一遍。

说实话，翻完之后我有个很强烈的感受：**大部分 Skill 都是 Slop（低质量垃圾），但少数写得好的，也质量挺高。**

而且，质量比较高的 Skill 里面，用到的一些技巧和方法都无比的相似。

今天就把这些套路全拆出来，毫无保留地分享给大家。学完之后你写的 Skill，和那些 AI 随手生成的 Slop，会是完全不同的两个东西。

先说个前提：为什么 Skill 质量这么重要？

你可能会想，不就是一个 Markdown 文件嘛，写得好不好能差多少？

差太多了。

Skill 本质上是给 AI 的“操作手册”，你想想，你给新来的实习生一份写得模模糊糊的 SOP，和一份结构清晰、重点突出的 SOP，他的工作产出能一样吗？

同样的道理，一个写得好的 Skill，模型的输出质量、稳定性、一致性，跟写得烂的完全不在一个档次。

好，废话不多说，接下来上干货。

技巧一：渐进式加载——别把所有东西都塞进 SKILL.md

这是我认为最重要的一个技巧，没有之一。

Anthropic 官方叫它 **Progressive Disclosure**（渐进式披露），核心思想就一句话：**模型不需要的信息，就别让它看到。**

为什么？因为上下文窗口是公共资源。你的 SKILL.md 不是孤立存在的，它要和系统提示词、对话历史、用户输入、其他 Skill 的元数据一起挤在有限的上下文空间里。你塞得越多，留给真正工作的空间就越少。

具体怎么做？ 把详细内容拆到 [references/](#) 目录里，SKILL.md 只在需要的步骤才引用它。

拿我做的 code-review-expert 来说，四份 checklist 加起来好几千字——SOLID 原则检查清单、安全检查清单、代码质量检查清单、代码清理计划模板。如果全塞进 SKILL.md，一上来就吃掉大量上下文。

所以我把它们放在了 references/ 里：

```
code-review-expert/
├── SKILL.md                # 主文件，只有流程编排
└── references/
    ├── solid-checklist.md  # 第二步才加载
    ├── security-checklist.md # 第四步才加载
    ├── code-quality-checklist.md # 第五步才加载
    └── removal-plan.md     # 第三步才加载
```

SKILL.md 里只在对应步骤写一句 Load references/xxx.md，模型执行到那个步骤时才去读。

这个技巧在头部 Skill 里几乎是标配。

看看 baoyu-slide-deck（PPT 生成 Skill），它的 references 目录里有 15+ 个文件：

```
references/
├── analysis-framework.md  # 内容分析框架
├── base-prompt.md         # 基础 prompt 模板
├── design-guidelines.md   # 设计指南
├── layouts.md             # 布局选项
├── dimensions/            # 风格维度（质感、色调、排版、密度）
├── styles/                # 预设风格
└── config/                # 配置相关
```

如果这些全塞进 SKILL.md，那文件得好几千行。但现在 SKILL.md 只有几百行，核心流程清清楚楚，需要细节的时候再去加载。

说说反面教材。我翻到过一些 Skill，把 API 文档、使用示例、常见问题全塞在一个 SKILL.md 里，2000 多行。结果模型读完这一堆东西，真正重要的指令反而被淹没了。

记住一个原则：SKILL.md 控制在 500 行以内，超过的部分一律拆到 references/ 里。

技巧二：关键词轰炸 Description——决定你的 Skill 能不能被“看见”

frontmatter 里的 description 字段，可能是整个 Skill 里最容易被低估的部分。

很多人随手写一句“帮你做 Code Review”就完事了。但这个字段决定了两件事：

1. 模型什么时候自动触发你的 Skill
2. 用户在搜索时能不能找到你

写得好的 description 长什么样？

看看 ui-ux-pro-max 的 description，教科书级别的关键词轰炸：

```
description: "UI/UX design intelligence. 50 styles, 21 palettes,
50 font pairings, 20 charts, 8 stacks (React, Next.js, Vue, Svelte,
SwiftUI, React Native, Flutter, Tailwind). Actions: plan, build,
create, design, implement, review, fix, improve, optimize, enhance,
refactor, check UI/UX code. Projects: website, landing page,
dashboard, admin panel, e-commerce, SaaS, portfolio, blog, mobile app.
Styles: glassmorphism, claymorphism, minimalism, brutalism..."
```

这不是 description，这是一张“触发关键词网”。不管用户说“帮我设计一个 landing page”还是“优化一下这个 dashboard 的 UI”，都能命中。

再看 excalidraw-artist：

```
description: "Create beautiful, elegant Excalidraw diagrams based on
user intent. Use when user asks to draw, visualize, diagram, sketch,
illustrate concepts, create flowcharts, architecture diagrams, mind maps,
process flows, or any visual representation. Triggers on keywords like
'draw', 'diagram', 'visualize', 'sketch', 'flowchart', 'architecture',
'mind map', 'illustrate'."
```

注意它直接列出了触发关键词：“draw”、“diagram”、“visualize”、“sketch”——这些都是用户会说的自然语言。

对比一下写得差的：

```
# 差的 description
description: 代码审查工具 # 好的 description
description: "代码审查。当用户说'帮我 review'、'检查代码'、
'审查 PR'、'看看这段代码有没有问题'时使用。"
```

这里有个很重要的点：**description** 里的“何时使用”信息不要放到 **SKILL.md** 正文里。因为正文只有在 Skill 触发之后才会被加载，所以你在正文里写“When to Use This Skill”对触发完全没用。skill-creator 官方文档里专门强调了这一点。

技巧三： workflow 清单模式——给模型一条清晰的路

这个技巧的效果非常直观：用了和没用，模型的执行一致性天差地别。

核心思路就是在 SKILL.md 里提供一个可追踪的 checklist，让模型按步骤打勾。

baoyu-slide-deck 这个 Skill 的做法我觉得可以借鉴：

Copy this checklist and check off items as you complete them: Slide Deck Progress: []

Step 1: Setup & Analyze

- [] 1.1 Load preferences
- [] 1.2 Analyze content
- [] 1.3 Check existing ⚠️ REQUIRED
- [] Step 2: Confirmation ⚠️ REQUIRED
- [] Step 3: Generate outline
- [] Step 4: Review outline (conditional)
- [] Step 5: Generate prompts
- [] Step 6: Review prompts (conditional)
- [] Step 7: Generate images
- [] Step 8: Merge to PPTX/PDF
- [] Step 9: Output summary

你看这里的设计，不只是简单列了几个步骤。它做了几件很巧妙的事：

1. 用 ⚠️ 标记关键节点，告诉模型哪些步骤绝对不能跳过
2. 子步骤嵌套，复杂步骤拆得更细
3. 标注条件分支，比如 "Step 4: Review outline (conditional)" 意思是根据前面用户的选择决定是否执行

baoyu-cover-image 也是类似的做法，而且它更极端——第 0 步直接标了 🛑 BLOCKING：

- [] Step 0: Check preferences (EXTEND.md) 🛑 BLOCKING
- [] Found → load preferences → continue
- [] Not found → run first-time setup → MUST complete before Step 1

这就是在告诉模型："你要是敢跳过这步，后面全白费。"

我在 **code-review-expert** 里也用了类似的思路，但更偏向"递进式"的编排——先看改动范围，再看架构设计，然后看安全，最后看代码质量。每一步之间是从宏观到微观的递进关系，而不是平铺的。

这个设计是有讲究的。好的 Code Reviewer 不会上来就抠命名规范，而是先理解整体改动的意图。所以 Skill 的工作流也要符合这个逻辑。

没有 workflow 清单的 Skill 会怎样？ 模型会东一榔头西一棒子，安全性和命名规范混在一起说，重要的东西被不重要的淹没。

技巧四：用脚本封装确定性操作

有些操作是确定性的——不需要 AI 每次都"重新想"一遍怎么做，直接执行就好。这种操作应该封装成脚本，放在 **scripts/** 目录里。

ui-ux-pro-max 这个 Skill 的做法就特别典型。它把所有的设计知识——50 种 UI 风格、21 种配色方案、50 种字体组合——全塞进了一个本地数据库里，然后写了个 **search.py** 脚本：

```
python3 .claude/skills/ui-ux-pro-max/scripts/search.py "beauty spa" --domain color
```

模型不需要去"回忆"或者"编造"配色方案，直接查就行。结果精准、一致、不会幻觉。

baoyu-slide-deck 也用了脚本，但用途不同——它的 `merge-to-pptx.ts` 和 `merge-to-pdf.ts` 负责把生成的图片合并成 PPT 和 PDF。这种纯文件操作脚本跑比让模型一步步写命令靠谱得多。

什么时候该用脚本？ skill-creator 官方文档给了一个很好的判断标准：

| 当同样的代码被反复重写时，或者需要确定性可靠性时，就该用脚本。

拿 PDF 处理举例：旋转 PDF 页面这种操作，每次让模型从零写一段 Python 来处理，不如直接一个 `rotate_pdf.py` 脚本来得稳。

还有一个好处——脚本可以不加载进上下文就直接执行。模型只需要知道“有这么个脚本，参数是什么”，不需要把脚本内容读进来。这又省了一大块上下文空间。

技巧五：给模型“该问的问题”，而不是“该找的答案”

这个技巧是我做 code-review-expert 的时候悟出来的，效果好到超出预期。

一般人写 Skill 会这么写：

检查代码是否违反了单一职责原则（SRP）。

太笼统了。模型看到这句话，可能会泛泛地说“这个类做了太多事情”之类的废话。

但如果你换成一个具体的问题：

问自己：这个模块有几个不同的修改理由？

如果答案超过一个，它可能违反了 SRP。

效果完全不一样。模型会真正去分析代码，数一数“这个模块有几个修改理由”，然后给出具体的结论。

我在 security-checklist.md 里大量使用了这个技巧：

- 检查竞态条件不是说“注意竞态”，而是问：“两个请求同时打到这段代码会怎么样？”
- 检查边界条件不是说“处理好边界”，而是问：“如果这个值是 null / 是 0 / 是空数组，会怎样？”
- 检查 TOCTOU 漏洞不是说“注意 TOCTOU”，而是问：“在检查权限和实际操作之间，状态有没有可能被改变？”

为什么这招管用？ 因为模型特别擅长“带着问题找答案”。你给它一个好问题，它就能自己去代码里找到对应的片段，然后分析出答案。但如果你只给它一个抽象的指令，它往往不知道该把注意力放在哪里。

这个技巧不只适用于 Code Review，任何需要模型进行分析和判断的 Skill 都可以用。

技巧六：确认节点——别让模型自作主张

这个技巧在涉及“生成”或“修改”操作的 Skill 里特别重要。

核心思路：在关键操作之前，强制模型先停下来跟用户确认。

baoyu-cover-image 把这个做到了极致。它有两个确认节点：

- 1. **Step 0**  **BLOCKING**：加载用户偏好。如果是第一次用，必须完成偏好设置才能继续——不能跳过，不能默认。
- 2. **Step 2**  **REQUIRED**：确认五个维度（类型、配色、渲染风格、文字密度、情绪）。除非用户明确用了 `--quick` 参数，否则必须过这关。

baoyu-slide-deck 更复杂，它用了 AskUserQuestion 来做结构化确认，而且是两轮：

Round 1（必问）：风格、受众、数量、是否 review outline、是否 review prompts

Round 2（条件）：只有选了"自定义维度"才问——质感、色调、排版、密度

我在 code-review-expert 里也用了确认节点，但设计逻辑不太一样。review 结果出来之后，它不会自动去改代码，而是先问你：

要修全部？只修 P0/P1？修指定的？还是只看看不改？

这个"先 review 再确认"的设计是我特意做的。Code Review 的价值不只是发现问题，更重要的是让你理解问题。如果 AI 直接帮你改了，你连有什么问题都不知道，那 review 就没意义了。

反面教材：我见过一些 Skill，让 AI 一路自动执行到底，用户完全没有介入的机会。结果如果中间哪一步出了问题，后面全部白干。

技巧七：Pre-Delivery Checklist——交付前的最后一道防线

这个技巧特别适合输出型的 Skill（生成代码、生成设计、生成文档）。

ui-ux-pro-max 在最后加了一个交付前检查清单：

```
## Pre-Delivery Checklist ### Visual Quality [ ] No emojis used as icons (use SVG instead)
- [ ] All icons from consistent icon set
- [ ] Brand logos are correct
- [ ] Hover states don't cause layout shift ### Interaction [ ] All clickable elements have cursor-pointer
- [ ] Transitions are smooth (150-300ms) ### Accessibility [ ] All images have alt text
- [ ] Form inputs have labels
- [ ] prefers-reduced-motion respected
```

这份清单的精妙之处在于——每一条都是"具体的、可检查的"。不是"确保可访问性好"这种废话，而是"所有图片都有 alt text"，模型可以直接去验证。

code-review-expert 用的是另一种形式——P0 到 P3 的优先级分级：

等级	含义	怎么处理
P0	严重	必须 block merge
P1	高危	应该在合并前修复

等级 含义 怎么处理

P2 中等 建个 follow-up

P3 低优 可选优化

这也是一种"交付质量管控"——不是所有问题都同等重要，模型需要一个分级标准来判断什么该重点说、什么可以轻描淡写。

技巧八：参数系统——让 Skill 变成"可配置工具"

好的 Skill 不是一把锤子，是一个瑞士军刀。

看看 baoyu-slide-deck 的参数设计：

```
/baoyu-slide-deck content.md --style sketch-notes --audience executives --lang zh --slides 10
/baoyu-slide-deck content.md --outline-only
/baoyu-slide-deck content.md --prompts-only
/baoyu-slide-deck slide-deck/topic/ --images-only
/baoyu-slide-deck slide-deck/topic/ --regenerate 3
```

它支持局部重做（`--regenerate 3` 只重新生成第 3 张幻灯片）、分阶段执行（`--outline-only` 只生成大纲）、全自定义参数。这意味着用户可以根据需要灵活组合，而不是每次都从头到尾跑一遍。

baoyu-cover-image 同样出色，5 个维度的参数各自独立，可以任意组合：

```
/baoyu-cover-image article.md --type conceptual --palette warm --rendering flat-vector
/baoyu-cover-image article.md --quick # 跳过确认，全自动
/baoyu-cover-image article.md --ref style-ref.png # 带参考图
```

实现方式很简单——用 `$ARGUMENTS` 变量接收参数，然后在 SKILL.md 里说明解析规则：

```
## Options | Option | Description |
| ----- | ----- |
| `--style <name>` | 视觉风格 |
| `--quick` | 跳过确认 |
| `--ref <files>` | 参考图 |
```

同时别忘了在 frontmatter 里加 `argument-hint`，这样用户输入 `/` 的时候能看到参数提示：

```
argument-hint: [content] [--style name] [--quick]
```

技巧九：References 的分类组织——不是一股脑堆文件

当你的 references 文件变多之后，怎么组织就变得很重要了。

baoyu-cover-image 的 references 组织方式是我见过最讲究的：

```

references/
├─ palettes/           # 9 种配色方案，每种一个文件
├─ renderings/        # 6 种渲染风格，每种一个文件
├─ dimensions/        # 风格维度说明
│   └─ text.md
│   └─ mood.md
├─ config/             # 配置相关
│   └─ preferences-schema.md
│   └─ first-time-setup.md
│   └─ watermark-guide.md
├─ workflow/           # 工作流相关
│   └─ confirm-options.md
│   └─ prompt-template.md
├─ auto-selection.md   # 自动选择规则
├─ compatibility.md    # 兼容性矩阵
└─ types.md            # 封面类型说明

```

这种按"领域"分类的组织方式有一个巨大的好处：**模型只需要加载相关领域的文件。**

用户选了 warm 配色？只读 `palettes/warm.md`。选了 hand-drawn 渲染？只读 `renderings/hand-drawn.md`。不会一次性把 9 种配色方案全加载进来。

skill-creator 官方文档里管这叫 **Domain-specific organization**：

```

bigquery-skill/
├─ SKILL.md (overview and navigation)
└─ reference/
    ├─ finance.md      # 用户问财务指标时才加载
    ├─ sales.md        # 用户问销售数据时才加载
    └─ product.md      # 用户问产品指标时才加载

```

同样的思路，如果你的 Skill 支持多个框架，也可以这么组织：

```

cloud-deploy/
├─ SKILL.md
└─ references/
    ├─ aws.md          # 用 AWS 时才加载
    ├─ gcp.md          # 用 GCP 时才加载
    └─ azure.md        # 用 Azure 时才加载

```

关键原则：`references` 只嵌套一层，不要搞多层嵌套。所有 reference 文件都应该从 SKILL.md 直接引用，而且要清楚地说明什么时候该读哪个文件。

技巧十：CLI 工具 + Skill = MCP 替代方案

这个技巧不是写 SKILL.md 的技巧，而是 Skill 的一种设计模式——用 CLI 工具来替代 MCP Server。

最典型的例子就是 agent-browser。以前要让 AI 操作浏览器，你得启动 Playwright MCP Server，20+ 个工具的定义全塞进上下文。现在呢？一个 CLI + 一个 SKILL.md：


```
agent-browser open https://example.com
agent-browser snapshot -i
agent-browser click @e1
agent-browser fill @e2 "text"
```

SKILL.md 只需要列出命令参考，模型通过 Bash 直接调用。上下文消耗对比 MCP 减少高达 93%。

agent-browser 的 SKILL.md 很有意思，它在 frontmatter 里限制了工具权限：

```
allowed-tools: Bash(agent-browser:*)
```

这意味着这个 Skill 只能执行以 `agent-browser` 开头的 Bash 命令，不能乱执行其他东西。安全性拉满。

这个模式的核心逻辑是：把复杂的协议层（MCP）替换成简单的命令行接口（CLI），再用 Skill 来教模型怎么用这些命令。

以后如果你想给 AI 添加某种能力，先想想能不能写成 CLI 工具 + Skill，而不是直接上 MCP。

写在最后

如果你觉得上面技巧太多，不好记，可以在公众号回复 "skill" 领取我精心整理的可视化 Skill 技巧总结，可以随时快速翻阅。

回头看这些技巧，其实核心就三个字：**省、准、稳**。

- **省**：渐进式加载、references 分类——省上下文空间
- **准**：关键词轰炸、提问式指令——让模型知道该做什么
- **稳**：工作流清单、确认节点、交付前检查——保证输出质量

说到底，写 Skill 不是写文档，是在**设计一个 AI 工作流**。你越理解模型的特点（擅长带着问题找答案、容易在长文本里丢失重点、对结构化指令执行力更强），你写出来的 Skill 就越好用。

我自己 code-review-expert 的实现在这里，欢迎参考：

GitHub: <https://github.com/sanyuan0704/code-review-expert>

安装：`npx skills add sanyuan0704/code-review-expert`